GENRESEARCH-SERVIDOR: Análise de memória, Concorrência e E/S em uma Aplicação de IA Generativa.

SISTEMAS OPERACIONAIS - COMP0472 (2026.2 - AE)
Discente: Augusto César Honorato dos Santos
Docente: Prof. Dr. Glauco de Figueiredo Carneiro

---

1. Introdução e Contextualização do Problema

A ascensão da Inteligência Artificial (IA) generativa e das arquiteturas de Recuperação Aumentada por Geração (RAG - *Retrieval-Augmented Generation*) trouxe novos desafios para a engenharia de software e, consequentemente, para os Sistemas Operacionais. Diferente de aplicações web tradicionais, um pipeline de RAG é altamente intensivo em recursos: ele precisa gerenciar a leitura de grandes volumes de documentos, calcular e armazenar *embeddings* (vetores matemáticos), manter índices em memória, gerenciar filas de requisições e lidar com concorrência massiva.

O problema central que motiva este trabalho é que o mau gerenciamento desses recursos pelo Sistema Operacional (SO) ou pelo desenvolvedor pode levar a falhas críticas. Entre os principais gargalos, destacam-se a paginação excessiva (*thrashing*), as condições de corrida em memórias compartilhadas (caches) e o esgotamento de memória principal (RAM), que força o kernel a encerrar processos abruptamente através do *Out of Memory Killer* (OOM Killer).

Diante disso, este trabalho apresenta o **GenResearch**-Servidor, um serviço simplificado e observável, que simula as etapas de um pipeline de IA. O objetivo não é avaliar a qualidade semântica das respostas geradas, mas sim analisar empiricamente como o Linux gerencia a memória principal, a memória virtual, o armazenamento, as filas e a sincronização de threads sob uma carga de trabalho inspirada em IA.

---

2. Pergunta Norteadora

Como a memória principal, a memória virtual, o armazenamento, os caches, as filas compartilhadas e os mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?

---

3. Ambiente de Execução e Metodologia

Para garantir a reprodutibilidade e o isolamento dos testes, foi provisionada uma Máquina Virtual (VM) no Oracle VirtualBox. O ambiente foi rigorosamente documentado com as seguintes especificações:

- **Sistema Operacional:** Ubuntu 24.04.4 LTS (Noble Numbat).
- **Kernel:** Linux 6.8.0-134-generic (arquitetura x86_64).
- **Hardware Virtual:** 1 vCPU (x86_64) e 3.82 GB de Memória RAM.
- **Memória Swap:** 0 B (Zero bytes). A ausência de swap foi uma decisão metodológica intencional para permitir a observação do comportamento do SO sem uma "rede de segurança" no disco.
- **Ambiente de Desenvolvimento:** Python 3.12 isolado em ambiente virtual (venv), utilizando as bibliotecas psutil (para coleta de métricas do SO), pandas e matplotlib (para visualização de dados).

Conceitos Observados no Linux: Durante os testes, conceitos fundamentais de Sistemas Operacionais foram observados. O *MemFree* indicou a memória totalmente livre, enquanto o *MemAvailable* estimou a memória disponível para novas aplicações. O *Page Cache* (cache de páginas) mostrou-se essencial: o Linux manteve dados de arquivos na RAM mesmo após o fechamento do arquivo, acelerando leituras subsequentes e provando que essa memória não é "desperdiçada", mas sim gerenciada dinamicamente pelo kernel.

---

4. Arquitetura do GenResearch-Servidor

O serviço foi implementado em Python e dividido em módulos que simulam um RAG real:

1. **Ingestão e Armazenamento:** 500 arquivos de texto (.txt) sintéticos foram gerados. Para cada um, um embedding simulado de 384 dimensões foi criado (usando hash determinístico). Todos os dados foram persistidos em um banco de dados SQLite local (genresearch.db), simulando um índice vetorial.

2. **Fila de Requisições:** Utilização da estrutura queue.Queue (buffer limitado) para gerenciar a entrada de consultas de múltiplos produtores, distribuindo-as para múltiplos consumidores (workers).

3. **Recuperação e Geração:** O sistema calcula a similaridade de cosseno entre a query e os vetores no banco para achar o contexto. A "geração de texto" pelo LLM foi simulada com time.sleep, representando o custo computacional da inferência.

4. **Cache Compartilhado:** Um dicionário em memória centraliza as respostas já processadas para evitar retrabalho.

5. **Sincronização:** Uso de threading.Lock (Mutex) para proteger as regiões críticas do código (cache e contadores globais) contra condições de corrida.

---

5. Experimentos, Resultados e Análises

Para uma análise completa, a aplicação foi submetida a seis configurações experimentais (C1 a C6), repetidas múltiplas vezes.

5.1. Concorrência e Filas (C3)
Inicialmente, configurou-se um teste com 2 produtores e 2 consumidores, enviando 10 requisições para uma fila de capacidade 10. O sistema processou tudo em 2.99 segundos. Posteriormente, no experimento de estresse do servidor completo (C4), aumentou-se para 4 workers, uma fila de capacidade 20 e 50 requisições. O SO gerenciou perfeitamente o escalonamento dessas threads, concluindo o lote em 2.62 segundos. A utilização de uma fila limitada evitou o consumo descontrolado de RAM, o que ocorreria se as requisições fossem infinitamente enfileiradas.

5.2. O Perigo da Concorrência e a Solução com Mutex (Parte D)
Este foi um dos pontos mais críticos do estudo. Para o cache compartilhado, foram disparadas 5 threads simultâneas buscando processar a mesma consulta (query). Na versão sem sincronização, todas as threads leram o cache ao mesmo tempo, viram que estava vazio, e geraram a resposta simultaneamente. O resultado foi de 5 Cache Misses (desperdício de 100% do processamento), caracterizando uma clássica Condição de Corrida.

A correção foi aplicada inserindo um threading.Lock (Mutex) ao redor da verificação do cache. Agora, apenas uma thread por vez pode avaliar e inserir dados. O resultado mudou drasticamente para 1 Cache Miss e 4 Cache Hits. No experimento do servidor completo (C4), essa eficiência resultou em uma taxa de acerto (Hit Rate) de 56%, ou seja, mais da metade das requisições economizaram o "tempo de LLM".

5.3. Análise de Memória Virtual: mmap vs open/read (C1, C2 e Parte E)
A leitura de arquivos no Linux pode ser feita de forma convencional (open/read) ou por mapeamento de memória (mmap). Nos testes (C1 vs C2), a leitura convencional levou, em média, 0.05 segundos, enquanto o mmap levou 0.04 segundos. Em uma das rodadas a quente, o mmap foi até 4 vezes mais rápido.

Isso ocorre porque o mmap não copia os dados do disco para o buffer do processo. Ele cria um mapeamento na memória virtual e delega ao Page Cache do kernel a responsabilidade de buscar as páginas em disco apenas quando forem realmente acessadas. Para o SO, isso reduz a cópia de dados e o consumo de memória do processo (RSS), algo vital para carregar índices de IA de vários gigabytes. O RSS do processo Python manteve-se estável em 14.02 MB durante as rodadas do benchmark, provando que o kernel gerenciou o cache de forma transparente.

5.4. Pressão de Memória e a Atuação do OOM Killer (C5)
Em um cenário ideal, a RAM nunca deve esgotar. Para testar o limite do SO, o script C5 alocou blocos de 500 MB em memória até preencher os 3.82 GB da VM. Com o uso da RAM ultrapassando 90.6% (3.46 GB usados), não havia partição swap para onde enviar as páginas menos utilizadas.

Neste momento, o kernel do Linux acionou o OOM Killer (Out of Memory Killer). O processo Python que estava causando a pressão de memória foi morto pelo SO com um sinal SIGKILL, resultando na mensagem Killed no terminal. Isso demonstra que o SO prioriza a estabilidade global do sistema em detrimento de um processo individual faminto por memória.

5.5. Falha Controlada e Robustez (C6)

Para evitar que o servidor trave inteiro quando há um pico de acessos, a aplicação deve saber lidar com filas cheias. No experimento C6, limitou-se a fila a 3 posições com um consumidor lento. Ao tentar enviar 6 itens de uma vez, o sistema capturou a exceção queue.Full para os itens excedentes. Ao invés de travar a aplicação, o código tratou o erro (simulando o retorno de um HTTP 503 - Service Unavailable no mundo real), garantindo a robustez do serviço.

---

6. Análise Aprofundada: Relação entre o Sistema Operacional e a IA Generativa

Os resultados obtidos com o GenResearch-Servidor evidenciam que a eficiência de uma aplicação de Inteligência Artificial não depende apenas do poder de processamento da CPU ou da GPU, mas fundamentalmente de como o Sistema Operacional gerencia os recursos de hardware.

O Page Cache do Linux mostrou-se um aliado invisível e poderoso. Em um cenário real de RAG, onde o índice vetorial (como FAISS ou ChromaDB) possui vários gigabytes, o mapeamento de memória (mmap) permite que o SO carregue apenas as partes do índice que estão sendo consultadas, mantendo o restante no disco. O SO atua como um "gerente de tráfego", decidindo o que fica na memória RAM (rápida) e o que vai para o armazenamento (lento), otimizando o custo-benefício do hardware.

Além disso, a gestão de concorrência provou ser um divisor de águas. Em aplicações de IA, a "geração" é o processo mais caro (exige muito processamento matricial). Se um cache falha por falta de sincronização (como demonstrado na condição de corrida), o servidor desperdiça ciclos valiosos de CPU recalculando a mesma resposta. A aplicação correta do Mutex (Lock) garantiu que o processamento fosse reaproveitado, simulando o comportamento de servidores de produção modernos.

Por fim, o comportamento do **OOM Killer** destaca a importância do design de infraestrutura. Em ambientes de nuvem (cloud), onde a memória é um recurso caro, a ausência de swap (ou sua má configuração) pode fazer com que um pico de requisições resulte no encerramento abrupto do servidor. O experimento provou que o kernel do Linux é implacável ao proteger o sistema como um todo, matando o processo que tenta ultrapassar os limites físicos da máquina.

---

7. Limitações do Estudo

É importante ressaltar que os resultados apresentados possuem limitações de escopo:

Simulação de Embeddings e Geração: Os vetores foram gerados via hash e a resposta do LLM foi simulada com time.sleep. Um modelo real (como Llama 3 ou GPT) consumiria não apenas muita RAM, mas também VRAM (memória de GPU), o que não foi avaliado nesta VM.

Ambiente de Máquina Única: Os testes refletem uma arquitetura monolítica. Em ambientes distribuídos (com múltiplos servidores e bancos de dados em rede), a latência de rede e a sincronização distribuída (ex: locks em bancos Redis) alterariam drasticamente as métricas de tempo.

Ausência de Swap: A VM foi configurada sem partição swap. Embora isso tenha forçado o OOM Killer rapidamente, em um servidor de produção, uma partição swap (mesmo que pequena) poderia ter mantido o processo vivo por mais tempo, às custas de lentidão extrema (thrashing).

Corpus Sintético: O corpus de 500 arquivos (2 MB) é pequeno para os padrões de IA. Um corpus de 50 GB exigiria uma análise de E/S de disco muito mais profunda e revelaria gargalos de armazenamento não vistos aqui.

---

8. Conclusão

O desenvolvimento e a execução do GenResearch-Servidor permitiram concluir, na prática, que os conceitos de Sistemas Operacionais são a espinha dorsal de qualquer aplicação de Inteligência Artificial. Longe de serem apenas disciplinas teóricas, tópicos como paginação, cache de páginas, condições de corrida, mutexes e filas de mensagens provaram-se fundamentais para o desempenho e a estabilidade do sistema.

A comparação entre open/read e mmap comprovou a eficiência do gerenciamento de memória virtual do Linux. A demonstração da condição de corrida e sua subsequente correção com Mutex evidenciaram a necessidade de um rigoroso controle de concorrência. Por fim, o teste de estresse com a atuação do OOM Killer mostrou que o Sistema Operacional é um guardião da integridade do hardware, agindo para evitar o colapso total da máquina.

Dessa forma, o objetivo da atividade foi plenamente alcançado: construiu-se uma aplicação simplificada, mas funcional e observável, cujas métricas coletadas (tempo, memória, cache hit rate) e artefatos gerados (gráficos, logs e código-fonte) comprovam a importância do alinhamento entre o desenvolvimento de software e o conhecimento profundo da arquitetura de Sistemas Operacionais.

---

9. Referências Bibliográficas

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. 9ª edição. Rio de Janeiro: LTC, 2015.
TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4ª edição. São Paulo: Pearson, 2016.
PYTHON SOFTWARE FOUNDATION. Documentação Oficial do Python 3.12. Disponível em: https://docs.python.org/3/. Acesso em: Outubro de 2026.

THE LINUX KERNEL ARCHIVES. Documentation: Memory Management (Page Cache and OOM Killer). Disponível em: https://www.kernel.org/doc/html/latest/. Acesso em: Outubro de 2026.